

AI-Powered Email Threat Detection, GeoLocation & Forensic Intelligence Platform

Build Documentation — SIH Problem Statement 26106 (AICTE, Cyber Security Cell)

This document breaks the problem statement into something buildable: architecture, module-wise TODOs, database/API design, ML approach, datasets, tools, references, and a sprint plan. Treat it as the team's single source of truth — update it as decisions change.

1. Problem Restated in Engineering Terms

Given a raw email (headers + body + attachments metadata), the system must:

Classify it — legitimate / suspicious / impersonated / phishing / BEC-fraud (NLP + ML).

Verify authenticity — parse and validate SPF, DKIM, DMARC, Return-Path, Received chain.

Trace origin — walk the `Received:` header chain to find the earliest untrusted hop, extract IP(s), geolocate them, fingerprint hosting/VPN/Tor/proxy.

Correlate — link sender domain/IP/aliases to known campaigns via a graph, and to threat intel (blacklists, WHOIS, prior incidents).

Report — surface fraud score + trace map + confidence in a dashboard, and generate a structured forensic report (with chain-of-custody logging).

Everything below maps to these five stages, which is also a clean way to split work across teammates:

NLP/Classification, Header Forensics, GeoIP/Infra Intel, Graph Correlation, Dashboard/Reporting.

2. High-Level Architecture



Ingestion Layer
.eml upload / IMAP fetch /
Gmail API / raw paste

Parsing Service
mail-parser / eml-parser
→ structured JSON (headers,
body, attachments, URLs)

NLP/ML Engine | Header Forensics | Origin & Infra |
- phishing classifier | - SPF/DKIM/DMARC | Intelligence |
- urgency/social | - spoofing anomaly | - GeolIP lookup |
engineering | detection | - VPN/Tor/hosting |
cue detection | fingerprinting |
- URL/attachment |
IOC extraction |

Correlation/Graph Engine (Neo4j or networkx+Postgres) | Risk Scoring Engine |
| (weighted fusion of all module outputs) |

Case & Alert Store
(Postgres + object store
for raw .eml evidence)



| API Layer (REST/GraphQL) |



| Dashboard (React) |
| - fraud score |
| - trace map (Leaflet) |
| - case management |
| - forensic report export |

Optional add-on (you already floated this): a **Chrome extension** that reads the currently-open Gmail message's raw headers via Gmail API, POSTs to the same backend `/analyze` endpoint, and renders a lightweight risk badge inline — reuses 100% of the backend, no separate logic needed.

3. Recommended Tech Stack

Layer	Choice	Why
Frontend	React + Vite, Tailwind, Leaflet/Mapbox for geo, D3 or Recharts for score/graph viz	Fast to build, good map + chart ecosystem
Backend API	FastAPI (Python)	Same language as ML stack, async, auto OpenAPI docs — huge time-saver for a hackathon
ML/NLP	scikit-learn or XGBoost for tabular fraud score; HuggingFace transformer (DistilBERT) or even TF-IDF+LogisticRegression if time-constrained, for phishing text classification	Ship a working baseline fast, upgrade if time allows
Header parsing	Python <code>mail-parser</code> , <code>email</code> stdlib, <code>eml_parser</code>	Battle-tested, handles malformed headers
Auth checks	<code>checkdmARC</code> , <code>pyspf</code> / <code>dkimpy</code>	Purpose-built libraries, don't reinvent SPF/DKIM parsing
GeoIP	MaxMind GeoLite2 (free, offline DB) or IPinfo.io API	GeoLite2 = no rate limits, works offline during demo
WHOIS/DNS	<code>python-whois</code> , <code>dnspython</code>	Standard
Threat intel	AbuseIPDB API, VirusTotal API (free tier), PhishTank feed	Free tiers sufficient for a hackathon-scale demo
Graph correlation	networkx (in-memory, simplest) → Neo4j if you want a "wow" graph visual and have time	Start simple, upgrade only if time permits
Database	PostgreSQL (cases, emails, indicators, users) + S3-compatible/local object storage for raw <code>.eml</code> evidence files	Relational fits case/alert data well
Queue/async	Celery + Redis, or FastAPI BackgroundTasks if scope is small	Analysis pipeline should not block the API request
Deployment	Docker Compose for local/demo; optionally free-tier Render/Railway for a live demo link	Judges love a live URL

4. Module-Wise Breakdown & TODOs

4.1 Ingestion & Parsing

☐ Accept `.eml` file upload (multipart) and raw paste of headers+body

- ☐ Add Gmail API OAuth flow for "connect inbox" demo mode (optional, high wow-factor)
- ☐ Parse into structured JSON: headers dict, body (plain+html), attachments list (name, hash, mime), extracted URLs
- ☐ Store raw `.eml` immutably (hash it — needed for chain-of-custody later)

4.2 Fraudulent Email Detection Engine (NLP/ML)

- ☐ Baseline: TF-IDF + Logistic Regression / XGBoost on subject+body → phishing probability
- ☐ Feature engineering: urgency keyword count, mismatched display-name vs email domain, presence of credential-request phrases, punctuation/caps anomalies, link-text vs actual-URL mismatch
- ☐ BEC-specific rules: invoice/payment-change keyword detection, executive-name spoofing check (compare display name to known org exec list)
- ☐ Obfuscated URL detection: unshorten links (bit.ly etc.), punycode/homoglyph domain check
- ☐ Output: category label + confidence score + list of triggered indicators (explainability matters for a forensic tool — don't just output a black-box score)

4.3 Email Header & Protocol Analysis

- ☐ Parse `Received:` chain into ordered hop list
- ☐ SPF check via `pyspf`/`checkdmARC` (pass/fail/none + alignment)
- ☐ DKIM signature validation via `dkimpy`
- ☐ DMARC policy + alignment result
- ☐ Flag anomalies: `Return-Path` domain ≠ `From` domain, missing/forged `Message-ID`, `Reply-To` mismatch, relay hops from suspicious ASNs
- ☐ Output a per-header "authenticity score"

4.4 Origin Traceability & Location Analysis

- ☐ From the Received-chain, identify the **first external hop** (the earliest hop not inside the recipient's trusted mail infra) — this is the real forensic target, not just the last "Received" line
- ☐ GeoIP lookup (MaxMind GeoLite2) → country/region/city/ISP
- ☐ Hosting/VPN/Tor detection: cross-check IP against known Tor exit node lists, hosting-provider ASN ranges (AWS/DigitalOcean/etc. ranges suggest disposable infra), public VPN IP lists
- ☐ WHOIS + DNS: domain age (freshly-registered domains are a strong phishing signal), registrar, MX record consistency with claimed sender org
- ☐ Render trace path on a map (Leaflet) — this is a strong demo visual, prioritize it

4.5 Identity Correlation & Attribution

- ☐ Build a graph: nodes = {sender domains, IPs, display names, attachment hashes}, edges = "seen together in email X"
- ☐ Cluster detection: same infrastructure reused across multiple flagged emails = campaign
- ☐ Cross-reference IP/domain against AbuseIPDB / VirusTotal / PhishTank
- ☐ Confidence-weighted attribution summary ("likely compromised legitimate account" vs "spoofed domain" vs "known malicious infra")

4.6 Risk Scoring Engine

- ☐ Weighted fusion function combining: NLP score, auth-check score, origin-anomaly score, threat-intel hits
- ☐ Keep it simple and explainable for the demo (a documented weighted sum beats an opaque ensemble you can't explain to judges under Q&A)

4.7 Dashboard & Case Management

- ☐ Inbox-style list view: sender, subject, risk score badge, timestamp
- ☐ Detail view: fraud score breakdown, header authenticity panel, geo trace map, triggered-indicator list
- ☐ Case/campaign grouping view (emails linked by shared infra)
- ☐ Forensic report export (PDF) — reuse the `/mnt/skills/public/pdf` skill when you get to this
- ☐ Alert feed (real-time-ish; polling or WebSocket) for high-risk emails

4.8 Privacy / Legal / Chain-of-Custody

- ☐ Hash + timestamp every ingested raw email (SHA-256) for evidentiary integrity
- ☐ Append-only audit log table (who viewed/exported which case, when)
- ☐ PII masking toggle for body content in shared/exported reports
- ☐ Configurable data retention window (even a simple cron-based purge is enough to demonstrate the concept)

5. Database Schema (PostgreSQL, first pass)

☐

sql

-- raw ingested email + hash for chain-of-custody

```
emails(  
  id UUID PK, raw_eml_path TEXT, sha256_hash TEXT,  
  received_at TIMESTAMPTZ, from_addr TEXT, display_name TEXT,  
  subject TEXT, ingested_at TIMESTAMPTZ  
)
```

```
headers_analysis(  
  email_id FK, spf_result TEXT, dkim_result TEXT, dmarc_result TEXT,  
  return_path TEXT, reply_to TEXT, message_id TEXT,  
  auth_score FLOAT, anomalies JSONB  
)
```

```
relay_hops(  
  id UUID PK, email_id FK, hop_order INT, ip_address TEXT,  
  hostname TEXT, timestamp TIMESTAMPTZ  
)
```

```
geo_data(  
  ip_address TEXT PK, country TEXT, region TEXT, city TEXT,  
  isp TEXT, asn TEXT, is_vpn BOOLEAN, is_tor BOOLEAN,  
  is_hosting BOOLEAN, updated_at TIMESTAMPTZ  
)
```

```
nlp_analysis(  
  email_id FK, category TEXT, confidence FLOAT,  
  triggered_indicators JSONB  
)
```

```
indicators (IOCs)(  
  id UUID PK, type TEXT, value TEXT, first_seen TIMESTAMPTZ,  
  threat_intel_hits JSONB  
)
```

```
email_indicators(  
  email_id FK, indicator_id FK -- many-to-many join  
)
```

```
cases(  
  id UUID PK, title TEXT, status TEXT, created_at TIMESTAMPTZ,  
  risk_score FLOAT  
)
```

```
case_emails(  
  case_id FK, email_id FK -- many-to-many join  
)
```

```
case_id FK, email_id FK
)

audit_log(
id UUID PK, user_id FK, action TEXT, target_id UUID,
timestamp TIMESTAMPTZ
)

users(
id UUID PK, name TEXT, role TEXT, org TEXT
)
```

Graph relationships (domain↔IP↔alias) can start as a simple `edges` table (`(node_a, node_b, relation_type, email_id)`) queried with recursive CTEs — no need for Neo4j unless there's time to spare.

6. API Design (FastAPI, first pass)

Endpoint	Method	Purpose
<code>/emails/upload</code>	POST	Upload <code>.eml</code> or paste raw headers+body
<code>/emails/{id}</code>	GET	Full parsed detail + all analysis results
<code>/emails/{id}/analyze</code>	POST	Trigger/re-run the full pipeline
<code>/emails/{id}/report</code>	GET	Generate forensic PDF report
<code>/cases</code>	GET/POST	List/create cases
<code>/cases/{id}</code>	GET	Case detail with linked emails + graph
<code>/indicators/{value}</code>	GET	IOC lookup across all cases
<code>/alerts</code>	GET	Real-time high-risk feed
<code>/gmail/connect</code>	GET	OAuth flow for live inbox demo mode

7. UI Screens

Login / Org selection (keep minimal — SSO not needed for a demo)

Dashboard home — alert feed, risk distribution chart, recent cases

Email inbox/list view — sortable by risk score

Email detail view — the centerpiece screen:

Header authenticity panel (SPF/DKIM/DMARC badges)

Geo trace map with hop markers + line path

NLP-flagged phrases highlighted in the email body

Risk score breakdown (radar or stacked bar chart)

Case/campaign view — graph visualization of linked infrastructure

Report export view — PDF preview + download

(Optional) **Chrome extension popup** — compact risk badge + "view full analysis" link into the dashboard

8. Datasets to Use / Train On

Dataset	Use
Nazario Phishing Corpus	Labeled phishing emails for classifier training
SpamAssassin Public Corpus	Ham vs spam baseline
Enron Email Dataset	Large legitimate-email corpus for negative examples / realistic headers
CEAS 2008 Spam Corpus	Additional labeled spam/phishing data
IWSPA-AP shared task dataset	Phishing detection benchmark, has header+body
PhishTank / OpenPhish live feeds	Real-time URL blacklist for IOC cross-referencing (not training, but live lookups)
MaxMind GeoLite2	Offline IP→geolocation database

Note: mix Enron (legit) with Nazario/CEAS (phishing) to get a balanced binary/multi-class training set. Keep a held-out test split and report precision/recall — judges will ask about false-positive rate, since that's the real-world pain point with email security tools.

9. Research Papers & References to Read

Phishing email detection using NLP/ML — search recent (2022–2025) survey papers on IEEE Xplore / arXiv for "phishing email detection machine learning survey" to get a feature-list and baseline-accuracy reference table you can cite.

SPF/DKIM/DMARC internals — RFC 7208 (SPF), RFC 6376 (DKIM), RFC 7489 (DMARC). Reading the actual RFCs (or a good summary) will help you correctly explain *why* an alignment failure matters, which matters a lot in Q&A.

BEC (Business Email Compromise) detection literature — look for papers specifically on "executive impersonation detection" and "invoice fraud email detection."

IP geolocation accuracy limitations — worth reading a paper/article on why IP geolocation is approximate (VPN/proxy/CGNAT), so you can honestly caveat this in your pitch instead of overclaiming precision.

Graph-based threat attribution — search "threat intelligence graph correlation" for how SOC tools link IOCs into campaigns; this will directly inform your edges table design.

MITRE ATT&CK (Email-based initial access techniques, e.g. T1566 Phishing) — useful vocabulary to map your detections to, which SIH judges from a cybersecurity background will recognize.

(Use web search/Google Scholar with these phrases when you sit down to read — happy to pull and summarize specific papers if you want, just point me at the topic.)

10. External Tools & APIs (mostly free-tier friendly)

MaxMind GeoLite2 — free offline GeoIP database

AbuseIPDB — free-tier IP reputation API

VirusTotal — free-tier URL/file/domain reputation API

PhishTank — free phishing URL feed

checkmarc / pyspf / dkimpy — Python libraries for auth checks

python-whois, dnspython — domain/DNS intel

Tor exit node list (check.torproject.org publishes it) — for Tor-exit flagging

11. Suggested Build Order (phase-based, not date-based — slot into whatever time you actually have)

Phase 1 — Core pipeline (get *something* end-to-end working first)

- ☐ `.eml` upload → parse → store
- ☐ SPF/DKIM/DMARC check → authenticity score
- ☐ Basic NLP classifier (even TF-IDF+LogReg) → fraud category
- ☐ GeoIP lookup on first external hop
- ☐ Single "email detail" page showing all of the above — this alone is a demoable MVP

Phase 2 — Forensic depth

- ☐ Full Received-chain reconstruction + anomaly flags
- ☐ WHOIS/DNS domain-age checks
- ☐ VPN/Tor/hosting fingerprinting
- ☐ Threat-intel API cross-referencing

Phase 3 — Correlation & case management

- ☐ Graph/edges table + campaign clustering
- ☐ Case management view
- ☐ Alert feed

Phase 4 — Polish for demo

- ☐ Forensic PDF report export
- ☐ Chrome extension (if time allows — big wow-factor, reuses backend)
- ☐ Seed the demo with a curated set of realistic sample emails (mix of clean, phishing, BEC, spoofed) so the live demo is reliable and not dependent on live internet mail flow

12. Suggested Role Split (for however many teammates you have)

Backend/API owner — FastAPI, DB schema, pipeline orchestration

ML/NLP owner — classifier training, feature engineering, dataset prep

Forensics owner — header parsing, SPF/DKIM/DMARC, GeoIP/WHOIS/threat-intel integration

Frontend owner — dashboard, maps, charts, report view

One person should own **the demo script and slide deck** end-to-end so the pitch narrative doesn't get assembled at the last minute

13. Demo Script Outline (for the pitch)

Open with the problem: a realistic BEC/phishing email that "looks fine" to a human

Upload it live → dashboard shows fraud score + why (SPF fail, spoofed domain, urgency language highlighted)

Click into the geo trace map → show the hop path and flagged VPN/hosting origin

Show the case view → this email is linked to 2 other flagged emails via shared infrastructure (this is the "forensic intelligence," not just detection — emphasize it, it's what differentiates you from a spam filter)

Export the forensic report PDF

(If built) show the Chrome extension flagging a live Gmail message

Close on privacy/chain-of-custody handling — judges from AICTE Cyber Security Cell will care about this being more than a toy

14. Risks & Honest Caveats (be upfront about these, don't let judges catch you first)

IP geolocation is **approximate** (city-level accuracy is not guaranteed, VPN/CGNAT breaks it) — frame outputs as "probable origin," not certain

A classifier trained on public datasets (Enron/Nazario/etc.) may not generalize perfectly to novel attack styles — mention planned continuous retraining

Real inbox integration (Gmail API) requires OAuth scopes and review for production use — fine for a demo with your own test account, call out the compliance path for real deployment

Chain-of-custody and legal evidentiary standards vary by jurisdiction — position this as "supports investigation," not as a legal certification tool

Keep this file updated as you make decisions — especially Section 4 (TODOs) and Section 11 (phase progress). It's meant to be a living doc, not a one-time spec.